An Analysis of Perceptron

Perceptron

$$y = \operatorname{sign}(w \cdot x + b)$$

$$w \leftarrow w + \eta(y - \hat{y})x$$

Perceptron -- Part 1

Further reading Aizerman, M. A. and Braverman, E. M. and Lev I. Rozonoer. Theoretical foundations of the potential function method in pattern recognition learning. Automation and Remote Control, 25:821–837, 1964. Rosenblatt, Frank (1958), The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain, Cornell Aeronautical Laboratory, Psychological Review, v65, No. 6, pp. 386–408. doi:10.1037/h0042519. Rosenblatt, Frank (1962), Principles of Neurodynamics. Washington, DC: Spartan Books. Minsky, M. L. and Papert, S. A. 1969. Perceptrons. Cambridge, MA: MIT Press. Gallant, S. I. (1990). Perceptron-based learning algorithms. IEEE Transactions on Neural Networks, vol. 1, no. 2, pp. 179–191. Olazaran Rodriguez, Jose Miguel. A historical sociology of neural network research. PhD Dissertation. University of Edinburgh, 1991. Mohri, Mehryar and Rostamizadeh, Afshin (2013). Perceptron Mistake Bounds arXiv:1305.0208, 2013. Novikoff, A. B. (1962). On convergence proofs on perceptrons. Symposium on the Mathematical Theory of Automata, 12, 615–622. Polytechnic Institute of Brooklyn. Widrow, B., Lehr, M.A., "30 years of Adaptive Neural Networks: Perceptron, Madaline, and Backpropagation," Proc. IEEE, vol 78, no 9, pp. 1415–1442, (1990). Collins, M. 2002. Discriminative training methods for hidden Markov models: Theory and experiments with the perceptron algorithm in Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP '02). Yin, Hongfeng (1996), Perceptron-Based Algorithms and Analysis, Spectrum Library, Concordia University, Canada

Perceptron -- Part 2

d j ∈ { 0 , 1 } {\displaystyle d_{j}\in \{0,1\}} is the desired output value of the perceptron for that input. We show the values of the features as follows:

Perceptron -- Part 3

Boolean function When operating on only binary inputs, a perceptron is called a linearly separable Boolean function, or threshold Boolean function. The sequence of numbers of threshold Boolean functions on n inputs is OEIS A000609. The value is only known exactly up to n = 9 {\displaystyle n=9} case, but the order of magnitude is known quite exactly: it has upper bound 2 n 2 − n log 2 ■ n + O (n) {\displaystyle 2^{n^{2}-n\log _{2}n+O(n)}} and lower bound 2 n 2 − n log 2 ■ n − O (n) {\displaystyle 2^{n^{2}-n\log _{2}n-O(n)}} . Any Boolean linear threshold function can be implemented with only integer weights. Furthermore, the number of bits necessary and sufficient for representing a single integer weight parameter is Θ (n ln ■ n) {\displaystyle \Theta (n\ln n)} .

Perceptron -- Part 4

The following simple proof is due to Novikoff (1962). The idea of the proof is that the weight vector is always adjusted by a bounded amount in a direction with which it has a negative dot product, and thus can be bounded above by O(√t), where t is the number of changes to the weight vector. However, it can also be bounded below by O(t) because if there exists an (unknown) satisfactory weight vector, then every change makes progress in this (unknown) direction by a positive amount that depends only on the input vector.

Perceptron -- Part 5

Although the perceptron initially seemed promising, it was quickly proved that perceptrons could not be trained to recognise many classes of patterns. This caused the field of neural network research to stagnate for many years, before it was recognised that a feedforward neural network with two or more layers (also called a multilayer perceptron) had greater processing power than perceptrons with one layer (also called a single-layer perceptron). Single-layer perceptrons are only capable of learning linearly separable patterns. For a classification task with some step activation function, a single node will have a single line dividing the data points forming the patterns. More nodes can create more dividing lines, but those lines must somehow be combined to form more complex classifications. A second layer of perceptrons, or even linear nodes, are sufficient to solve many otherwise non-separable problems. In 1969, a famous book entitled Perceptrons by Marvin Minsky and Seymour Papert showed that it was impossible for these classes of network to learn an XOR function. It is often incorrectly believed that they also conjectured that a similar result would hold for a multi-layer perceptron network. However, this is not true, as both Minsky and Papert already knew that multi-layer perceptrons were capable of producing an XOR function. (See the page on Perceptrons (book) for more information.) Nevertheless, the often-miscited Minsky and Papert text caused a significant decline in interest and funding of neural network research. It took ten more years until neural network research experienced a

resurgence in the 1980s. This text was reprinted in 1987 as "Perceptrons - Expanded Edition" where some errors in the original text are shown and corrected.

Perceptron -- Part 6

Multiclass perceptron Like most other techniques for training linear classifiers, the perceptron generalizes naturally to multiclass classification. Here, the input x {\displaystyle x} and the output y {\displaystyle y} are drawn from arbitrary sets. A feature representation function f (x , y) {\displaystyle f(x,y)} maps each possible input/output pair to a finite-dimensional real-valued feature vector. As before, the feature vector is multiplied by a weight vector w {\displaystyle w} , but now the resulting score is used to choose among many possible outputs:

Perceptron -- Part 7

A Perceptron implemented in MATLAB to learn binary NAND function Archived 2012-11-06 at the Wayback Machine Chapter 3 Weighted networks - the perceptron and chapter 4 Perceptron learning of Neural Networks - A Systematic Introduction by Raúl Rojas (ISBN 978-3-540-60505-8) History of perceptrons Mathematics of multilayer perceptrons Applying a perceptron model using scikit-learn - https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Perceptron.html

Perceptron -- Part 8

Learning a Boolean function Consider a dataset where the x {\displaystyle x} are from { − 1 , + 1 } n {\displaystyle \{-1,+1\}^{n}} , that is, the vertices of an n-dimensional hypercube centered at origin, and y = θ (x i) {\displaystyle y=\theta (x_{i})} . That is, all data points with positive x i {\displaystyle x_{i}} have y = 1 {\displaystyle y=1} , and vice versa. By the perceptron convergence theorem, a perceptron would converge after making at most n {\displaystyle n} mistakes. If we were to write a logical program to perform the same task, each positive example shows that one of the coordinates is the right one, and each negative example shows that its complement is a positive example. By collecting all the known positive examples, we eventually eliminate all but one coordinate, at which point the dataset is learned. This bound is asymptotically tight in terms of the worst-case. In the worst-case, the first presented example is entirely new, and gives n {\displaystyle n} bits of information, but each subsequent example would differ minimally from previous examples, and gives 1 bit each. After n + 1 {\displaystyle n+1} examples, there are 2 n {\displaystyle 2n} bits of information, which is sufficient for the perceptron (with 2 n {\displaystyle 2n} bits of information). However, it is not tight in terms of expectation if the examples are presented uniformly at random, since the first would give n {\displaystyle n} bits, the second n / 2 {\displaystyle n/2} bits, and so on, taking O (ln ■ n) {\displaystyle O(\ln n)} examples in total.